I-Logical Approaches Constraint Satisfaction Problems

Foundations of Neuro-Symbolic AI

Alex Goessmann

University of Applied Science Würzburg-Schweinfurt (THWS)

April 27, 2026

Constraint Satisfaction Problems

We call a hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ a constraint network, if

- each node $v \in \mathcal{V}$ is decorated by a variable X_v with leg dimension m_v.
- each edge is decorated by a constraint tensor $\tau^e [X_e]$ to each edge $e \in \mathcal{E}$ with leg dimensions m_e.

Given an assignment x_e to the variables of a constraint tensor, we say that the constraint is not satisfied, if $\tau^e [X_e = x_e] \neq 0$.

Constraint Satisfaction Problem (CSP)

Given such a decorated hypergraph $\mathcal{G}$, decide whether there is a state $x_\mathcal{V}$ such that all constraints are satisfied, that is $\tau^e [X_e = x_e] \neq 0$ for all edges.

Example: Satisfiability of a knowledge base

- Atomic formula correspond with variables X_v of dimension $m_v = 2$
- Formulas are the constraint tensors

Contraction Criterion

Recall: Contractions $\langle f \rangle_{[\varnothing]}$ count the number of satisfying assignments of the formula f. This is also true for more generic constraint networks.

Contraction criterion

A CSP is satisfiable, if and only if its contraction does not vanish, that is

$$\langle \{ \tau^{\mathsf{e}} [X_{\mathsf{e}}] \, : \, \mathsf{e} \in \mathcal{E} \} \rangle_{[\varnothing]} \neq 0 \, .$$

While global contraction is a generic solver, it cannot be an efficient one.

Outline

- Constraint Satisfaction Problems
- Sudoku
- Generalized unit propagation
- Graph Coloring
- Backtracking search

Example: Sudoku

Consider grids of $n^2 \times n^2$ cells assigned by numbers in $[n^2]$, where usually $n = 3$ but for visualization purposes we will take $n = 2$.

Constraints:

- Each cell contains exactly one digit
- Each digit appears exactly once in each row, column and square

Example of a $2^2 \times 2^2$ Sudoku with initial clues and unique solution:

0			
	1		
		3	
	2		

$=$

0	3	2	1
2	1	0	3
1	0	3	2
3	2	1	0

Representation of Sudoku

We enumerate each cell by tuples $r_0 r_1 c_0 c_1 \in [n]^4$, where (r_0, r_1) select the $r_0 \cdot n + r_1$th row and $(c_0 c_1)$ the $c_0 \cdot n + c_1$th column:

$$\begin{array}{cccc} 0000 & 0001 & 0010 & 0011 \\ 0100 & 0101 & 0110 & 0111 \\ 1000 & 1001 & 1010 & 1011 \\ 1100 & 1101 & 1110 & 1111 \end{array}$$

For each possible assignment i to each cell $r_0 r_1 c_0 c_1$ we define a hidden variable $X_{r_0 r_1 c_0 c_1 i}$, which is 1 if the cell $r_0 r_1 c_0 c_1$ is assigned by i and 0 else.

$$\{X_{r_0 r_1 c_0 c_1 i} \,:\, r_0 r_1 c_0 c_1 \in [n],\, i \in [n^2]\}$$

Sudoku constraint tensors

Sudoku constraints enforce "exactly-one" condition by tensors $\tau\left[X_{\mathcal{U}}\right]$ with indices

$$\tau\left[X_{\mathcal{U}} = x_{\mathcal{U}}\right] = \begin{cases} 1 & \text{if} \quad \sum_{u \in \mathcal{U}} x_u = 1 \\ 0 & \text{else} \end{cases}$$

We choose $\mathcal{U}$ differently for each constraint:

- **Position constraints:** $\mathcal{U} = \{(r_0 r_1 c_0 c_1 i) \,:\, i \in [n^2]\}$ for each cell $r_0 r_1 c_0 c_1$
- **Row constraints:** $\mathcal{U} = \{(r_0 r_1 c_0 c_1 i) \,:\, c_0, c_1 \in [n]\}$ for each row $r_0 r_1$ and each digit i
- **Column constraints:** $\mathcal{U} = \{(r_0 r_1 c_0 c_1 i) \,:\, r_0, r_1 \in [n]\}$ for each column $c_0 c_1$ and each digit i
- **Square constraints:** $\mathcal{U} = \{(r_0 r_1 c_0 c_1 i) \,:\, r_1, c_1 \in [n]\}$ for each square $r_0 c_0$ and each digit i

Decomposition of Sudoku constraints

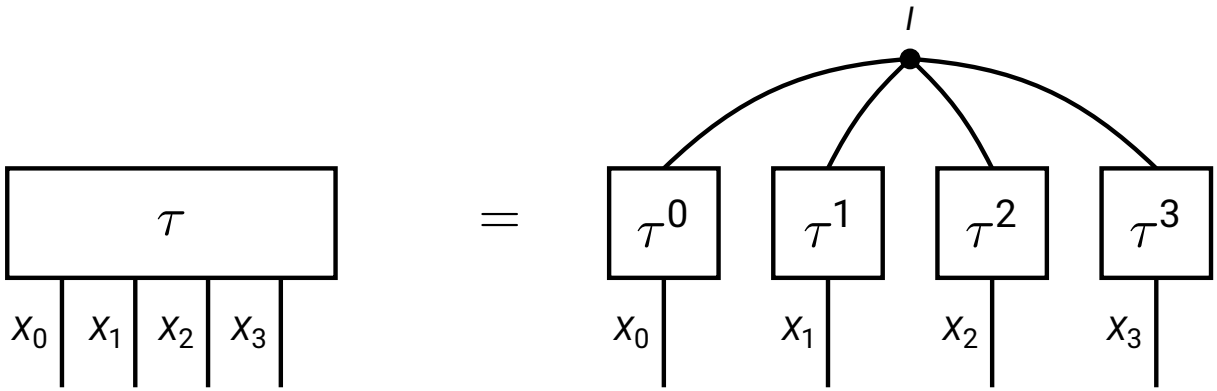
The constraints are decomposed using a decomposition variable $I \in [n^2]$ corresponding to the nonvanishing digit. This is enforce for each $i \in [n^2]$ by constraint matrices

$$\tau^i [X_i, I] = \epsilon_1 [X] \otimes \epsilon_i [I] + \epsilon_0 [X] \otimes (\mathbb{I} [I] - \epsilon_i [I]) \ .$$

For example:

$$\tau^2 [X, I] = \begin{pmatrix} 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} [X, I]$$

In total we have a decomposition (in the so-called CP-format):



Outline

- Constraint Satisfaction Problems
- Sudoku
- Generalized unit propagation
- Graph Coloring
- Backtracking search

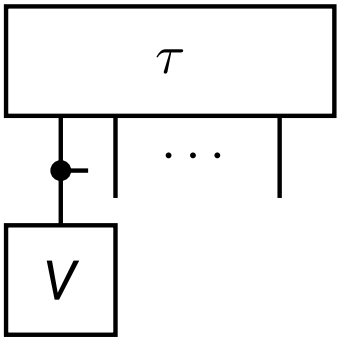
Disentangling the constraint network

Idea: When a CSP has a unique solution $x_{\mathcal{V}}$, then the contracted tensor is elementary:

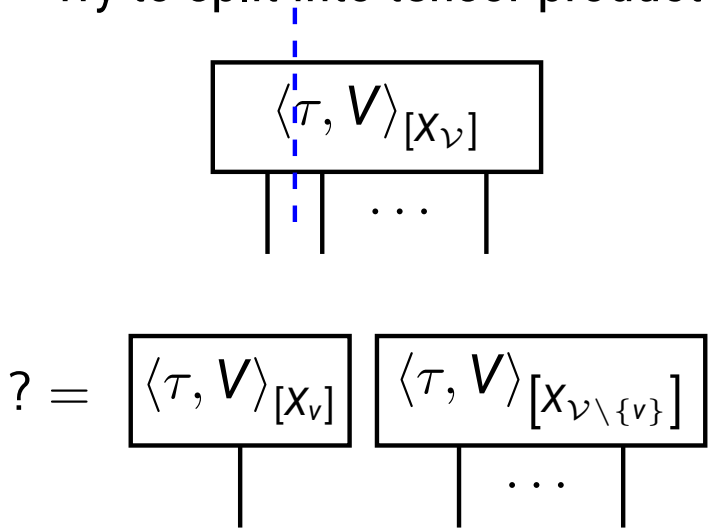
$$\langle \{\tau^{e}\,[X_e] \,:\, e \in \mathcal{E}\}\rangle_{[X_{\mathcal{V}}]} = \bigotimes_{v\in\mathcal{V}} \epsilon_{x_v}\,[X_v]$$

Aim: Perform manipulation operations to disentangle the constraints.

1.Step:
Contract vector on constraint

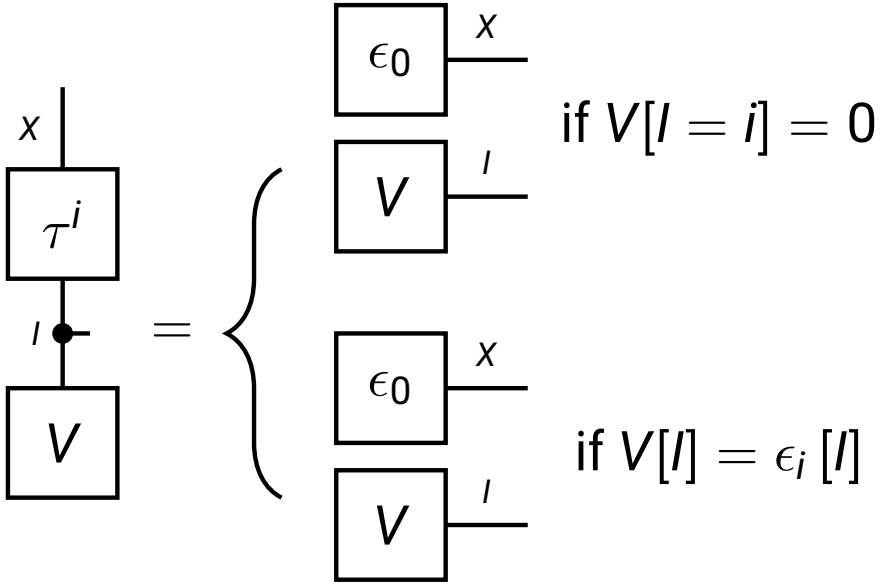


2.Step:
Try to split into tensor product

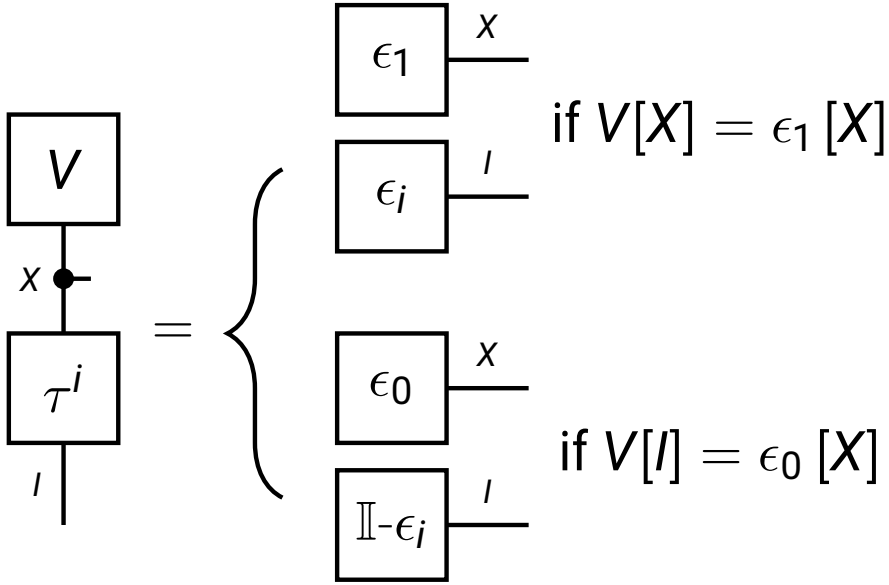


Disentangling properties of Sudoku matrices

Contraction along I:



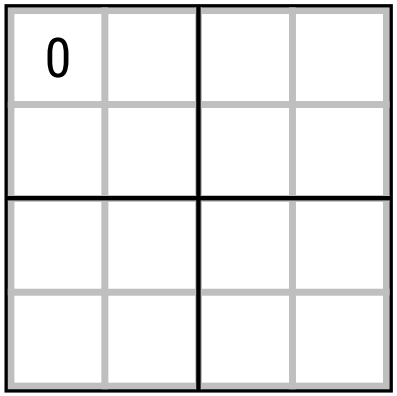
Contraction along X:

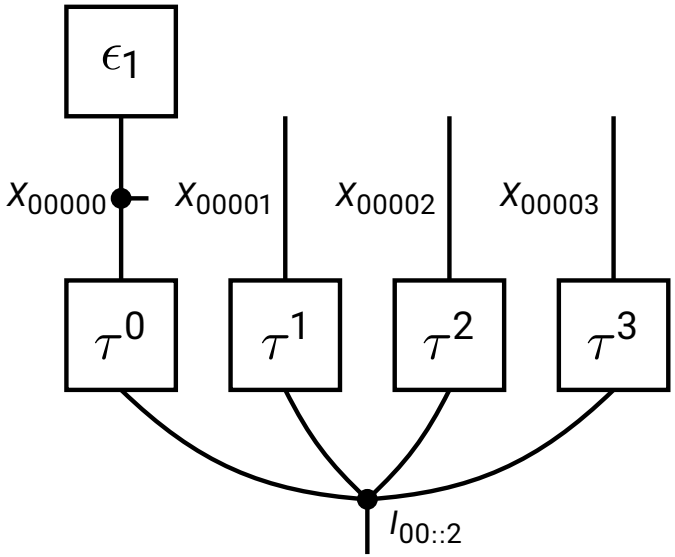


These disentangling matrix-vector contractions are exploited to solve (simple) Sudoku puzzles.

Disentangling a known position in Sudoku

Example: When 0 at position 0000, then no further digit can be assigned to that position.

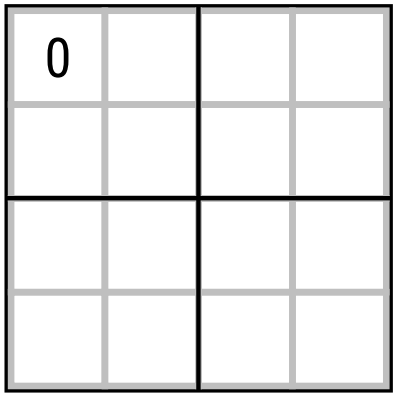


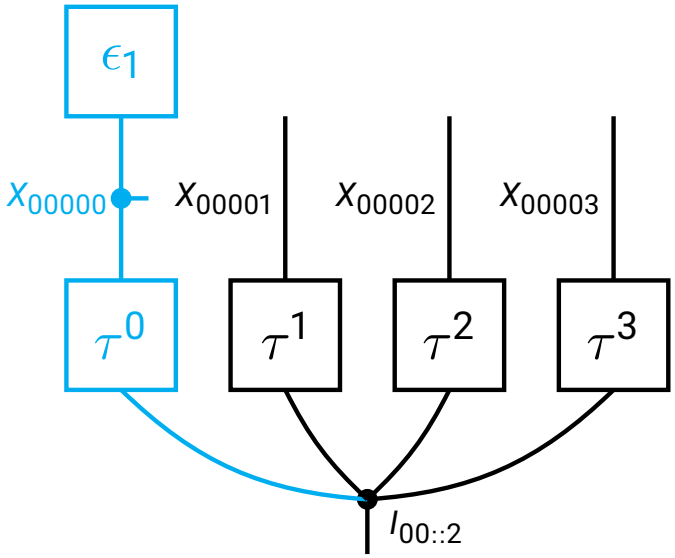


We disentangle each atomic variable corresponding with a known position.

Disentangling a known position in Sudoku

Example: When 0 at position 0000, then no further digit can be assigned to that position.

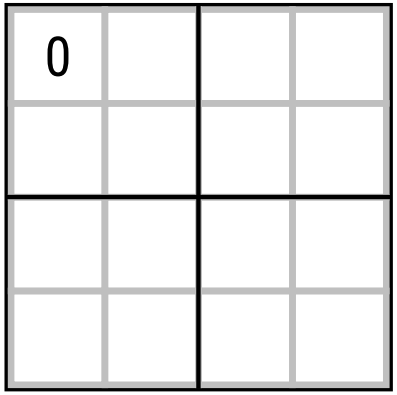


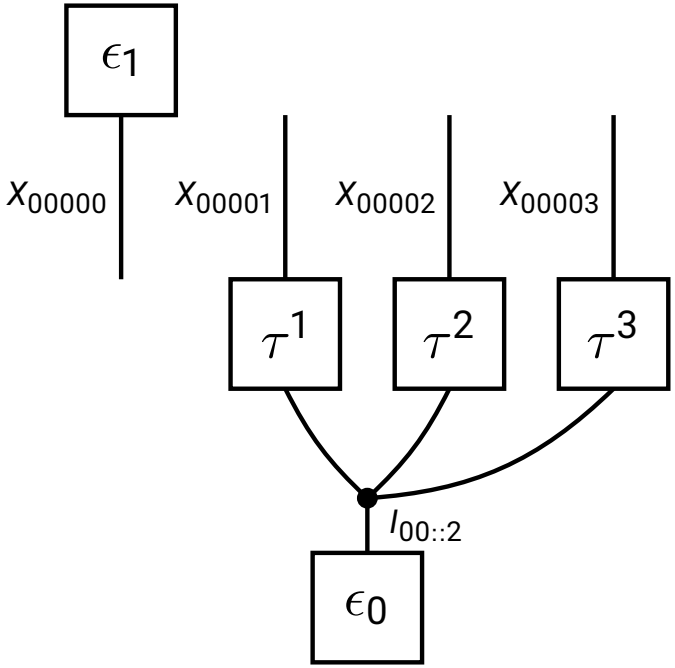


We disentangle each atomic variable corresponding with a known position.

Disentangling a known position in Sudoku

Example: When 0 at position 0000, then no further digit can be assigned to that position.

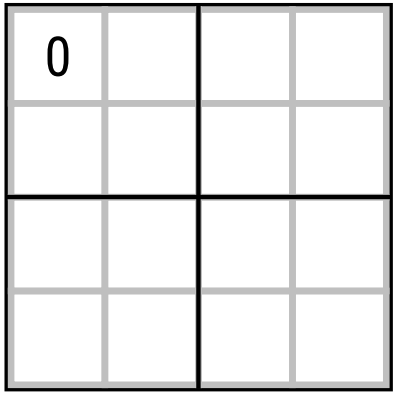


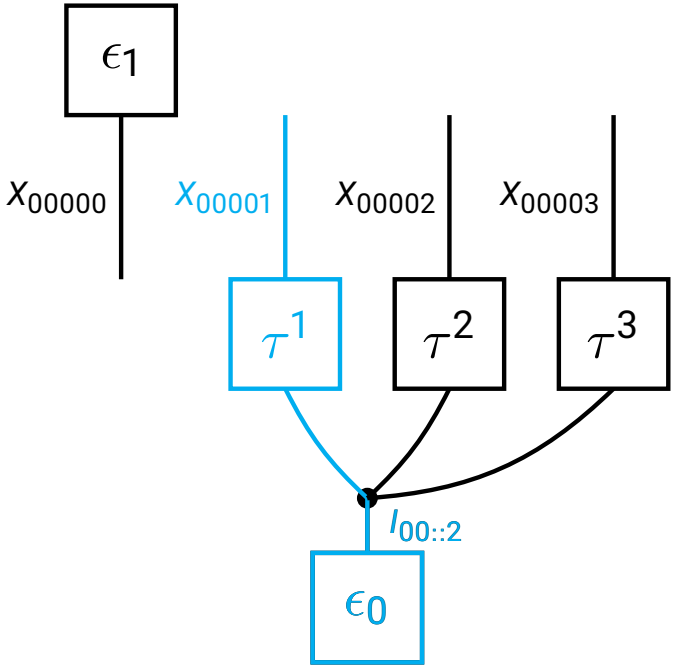


We disentangle each atomic variable corresponding with a known position.

Disentangling a known position in Sudoku

Example: When 0 at position 0000, then no further digit can be assigned to that position.

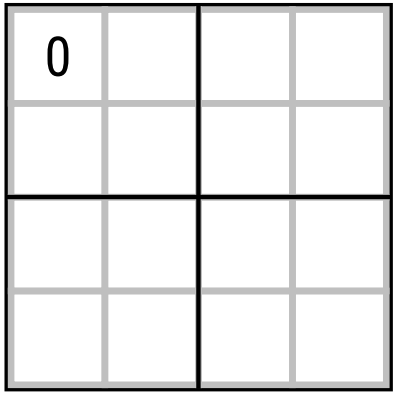


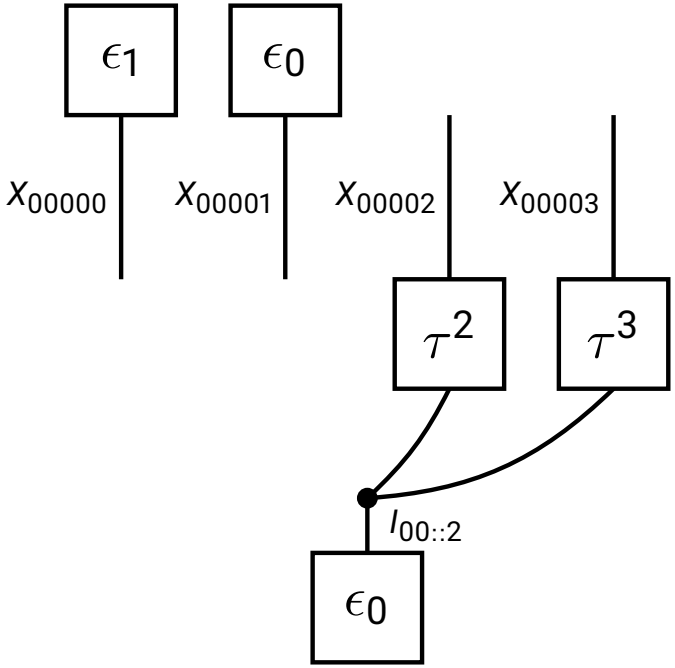


We disentangle each atomic variable corresponding with a known position.

Disentangling a known position in Sudoku

Example: When 0 at position 0000, then no further digit can be assigned to that position.

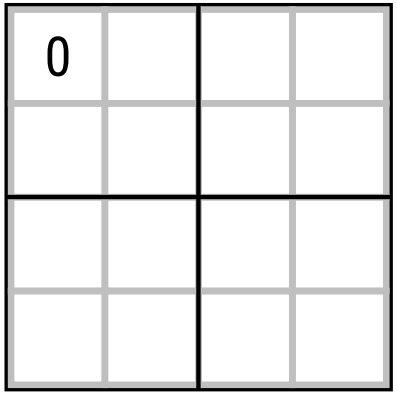


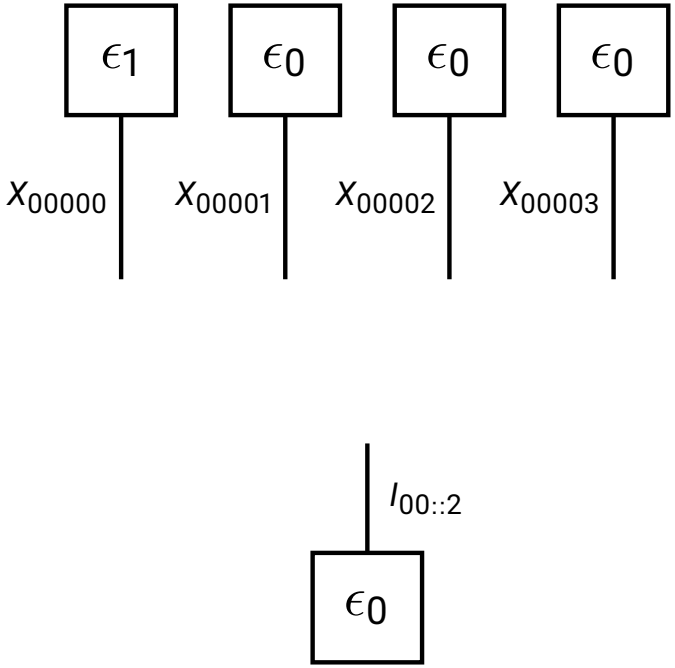


We disentangle each atomic variable corresponding with a known position.

Disentangling a known position in Sudoku

Example: When 0 at position 0000, then no further digit can be assigned to that position.





We disentangle each atomic variable corresponding with a known position.

Generalized unit propagation

Generalized unit propagation: Do until convergence

- Contract a vector to each neighboring tensor
- Try to split the resulting tensor into a vector and a tensor
- If success and the remaining tensor is a vector: Repeat for the remaining vector

Comparison with unit propagation so far:

- Works with arbitrary constraints (shape, values), while so far on unit clauses
- Performs sub-contraction and tries to split the resulting tensor

Generalized unit propagation algorithm

Require: Constraint network $\tau^{\mathcal{G}}$

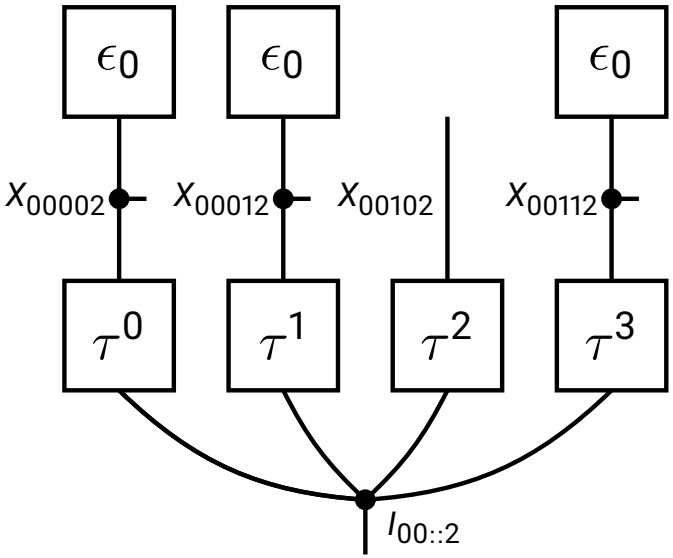
Ensure: Equivalent constraint network $\tau^{\mathcal{G}}$, with potentially less connectivity in $\mathcal{G}$

Initialize a queue Q of unary edges in $\mathcal{G}$
while Q not empty **do**
  $\{v\} = Q.pop()$
  for $e \in \mathcal{E} \ : \ v \in e$ **do**
    $\tau^{e}\left[X_e\right] \leftarrow \left\langle \tau^{e}\left[X_e\right], \tau^{\{v\}}\left[X_v\right] \right\rangle_{[X_e]}$
    Try to split $\tau^{e}\left[X_e\right]$ into a vector v and a tensor $e \backslash \{v\}$
    if Split successful and $|e \backslash \{v\}| = 1$ **then**
      $Q.push(e \backslash \{v\})$
    end if
  end for
end while
return $\tau^{\mathcal{G}}$

Matrix-vector example

Find the 2 in the first row:

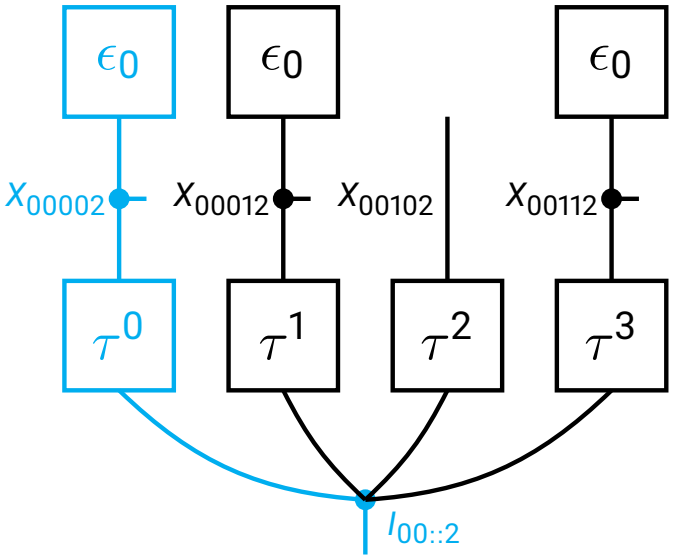
0	3		1



Matrix-vector example

Find the 2 in the first row:

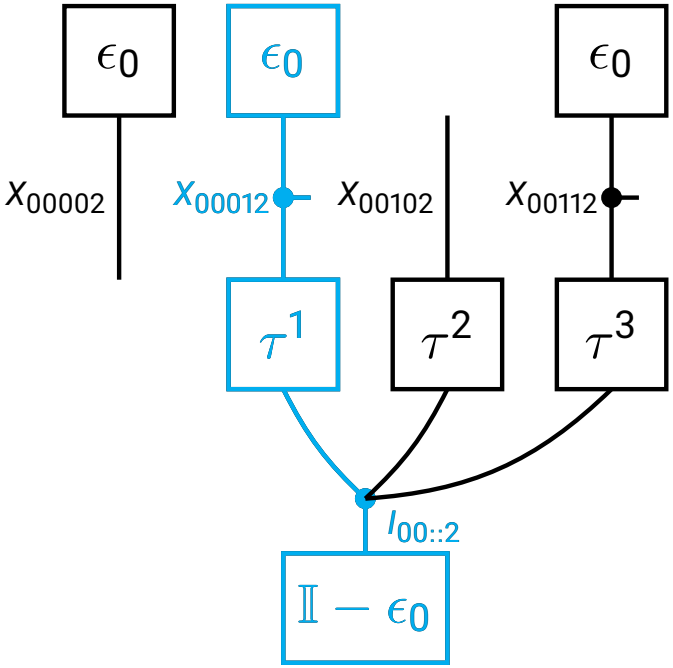
0	3		1



Matrix-vector example

Find the 2 in the first row:

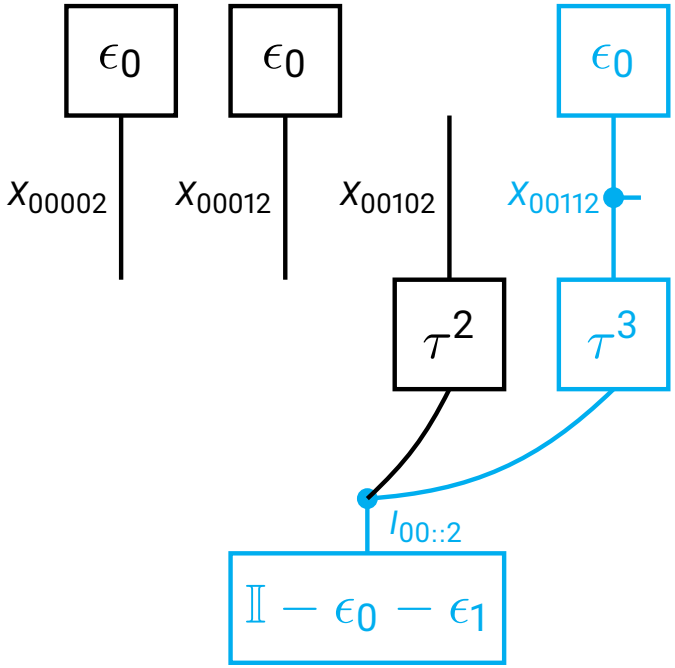
0	3		1



Matrix-vector example

Find the 2 in the first row:

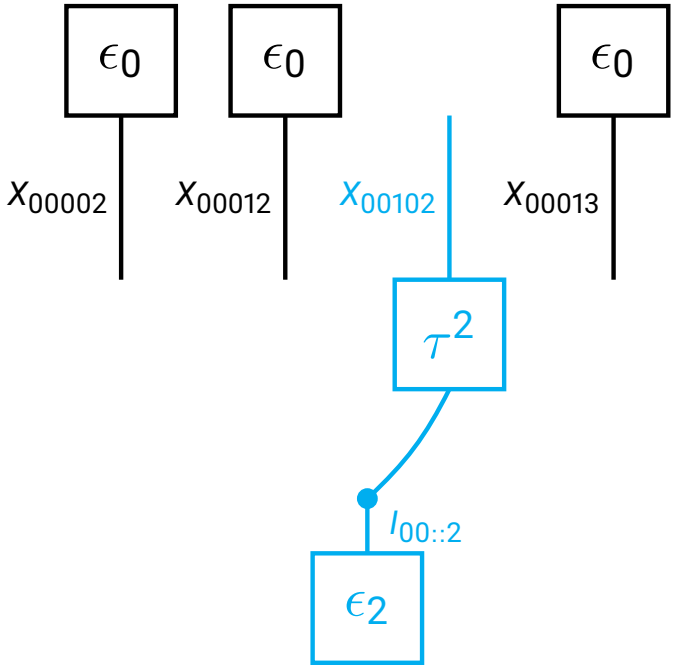
0	3		1



Matrix-vector example

Find the 2 in the first row:

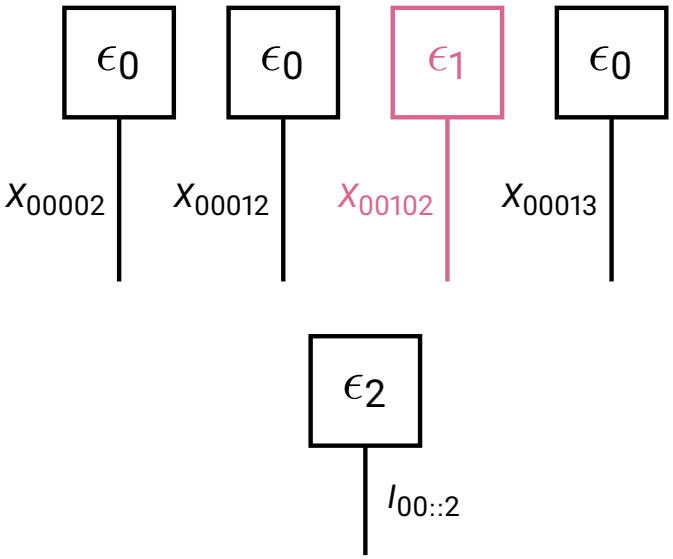
0	3		1



Matrix-vector example

Find the 2 in the first row:

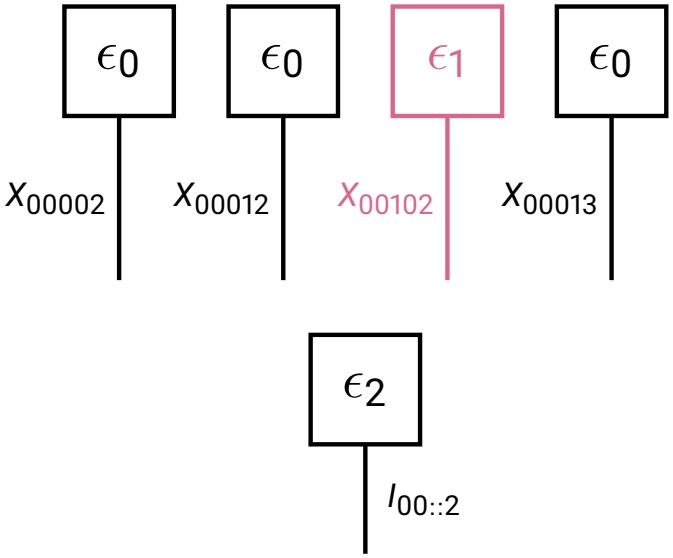
0	3	2	1



Matrix-vector example

Find the 2 in the first row:

0	3		1



With these matrix-vector contraction schemes, the example Sudoku can be solved:

0			
	1		
		3	
	2		

$=$

0	3	2	1
2	1	0	3
1	0	3	2
3	2	1	0

Generalized unit propagation

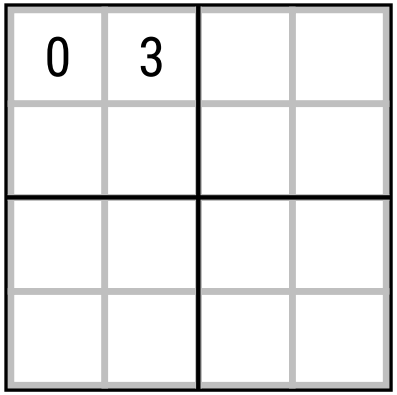
Same behaviour as classical Sudoku solvers:

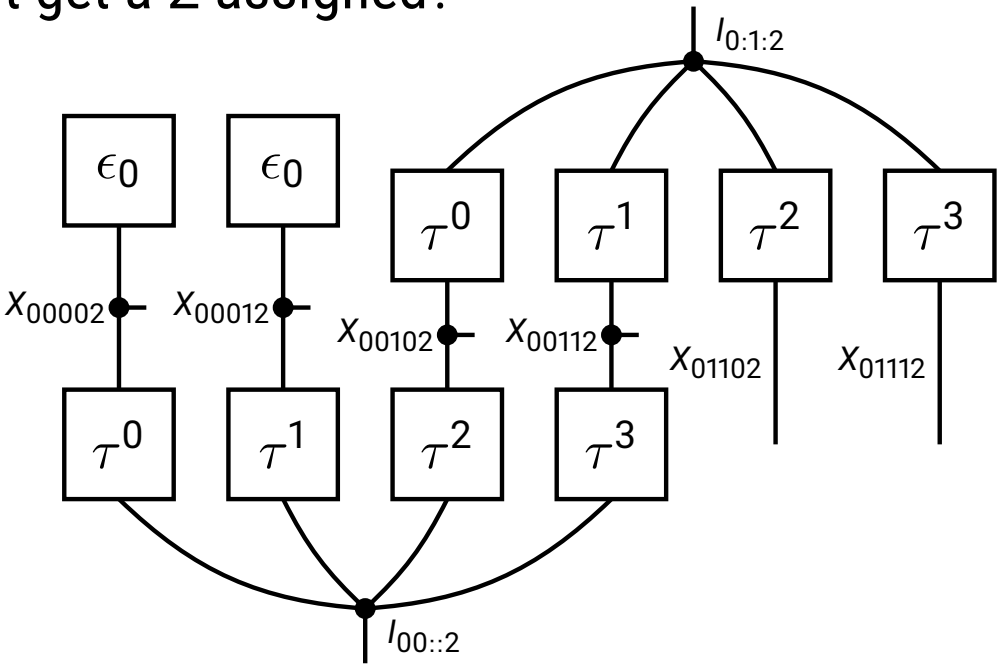
- Dancing Links [Knuth: Art of Computer Programming, Vol. 4b]
- AC-3 [Mackworth: Consistency in Networks of Relations]

But: Often convergence without solution. On the Nicoli100 benchmark: 49% solved by generalized unit propagation.

Limitations

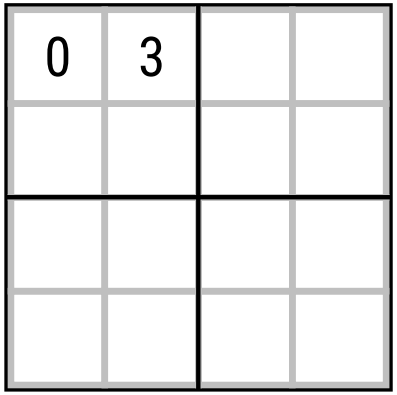
Example: Which cells cannot get a 2 assigned?

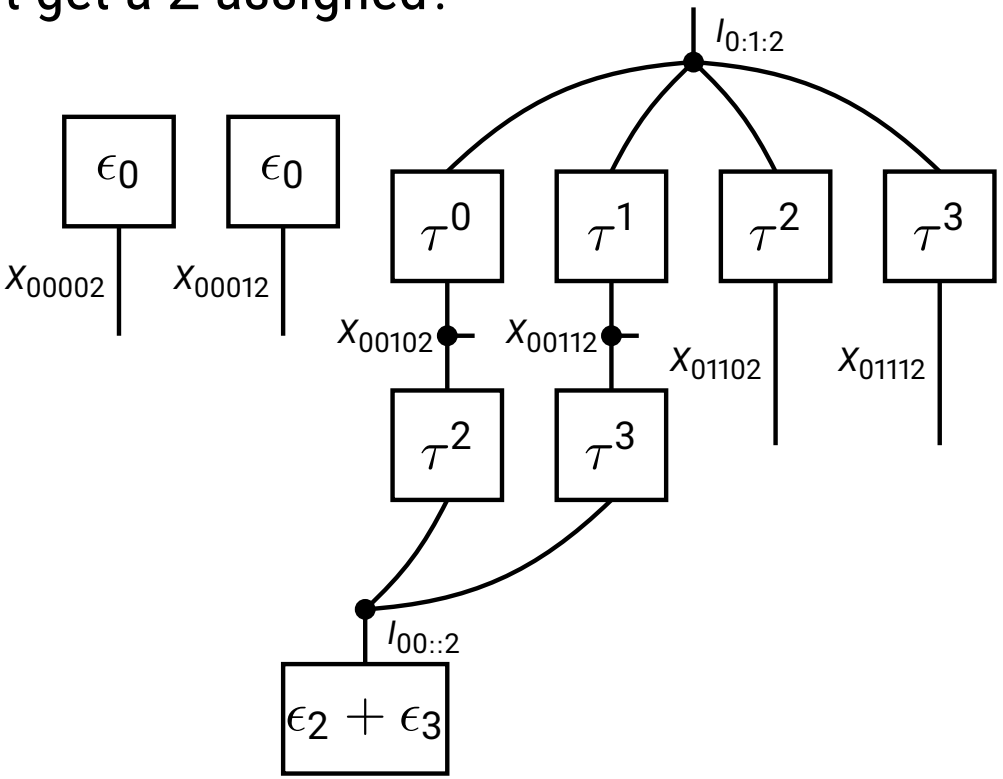




Limitations

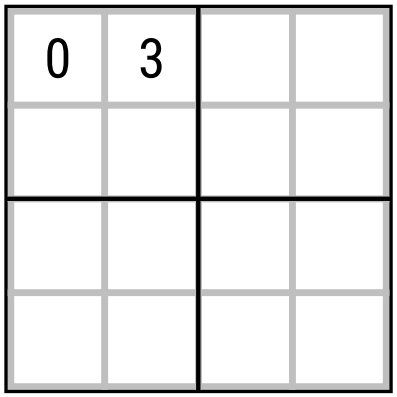
Example: Which cells cannot get a 2 assigned?

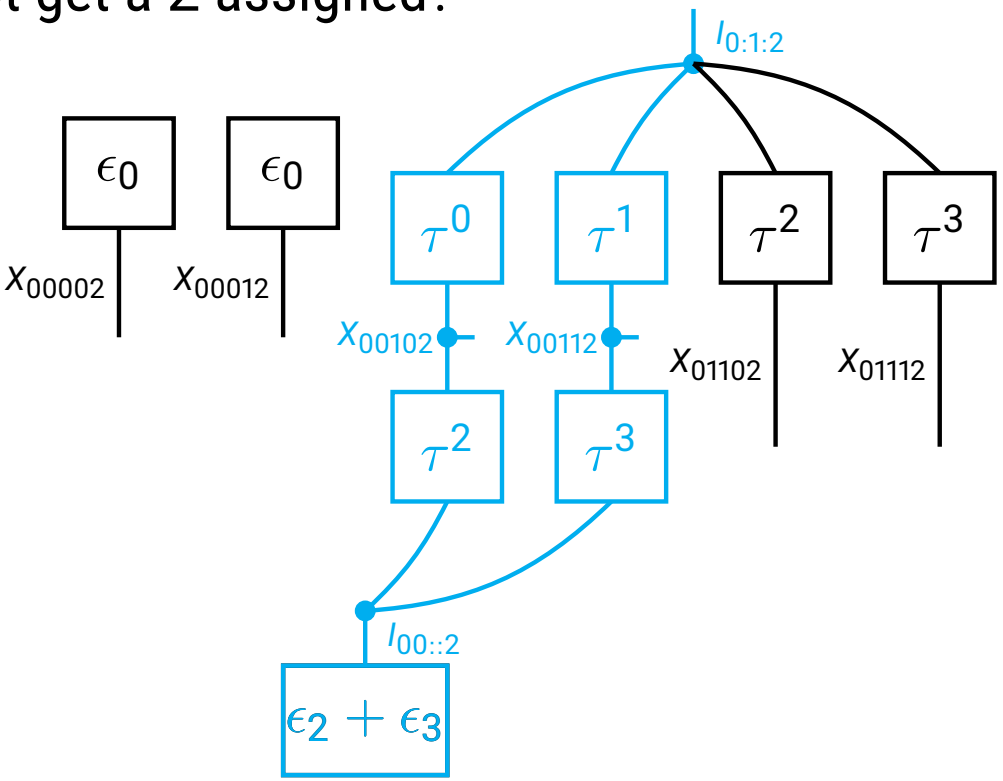




Limitations

Example: Which cells cannot get a 2 assigned?

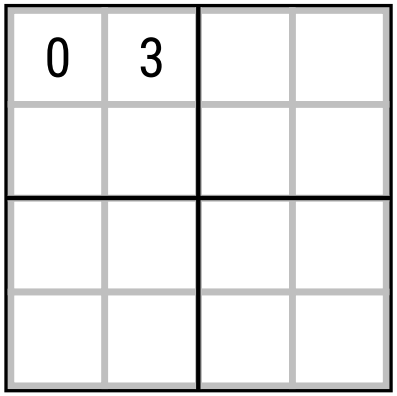


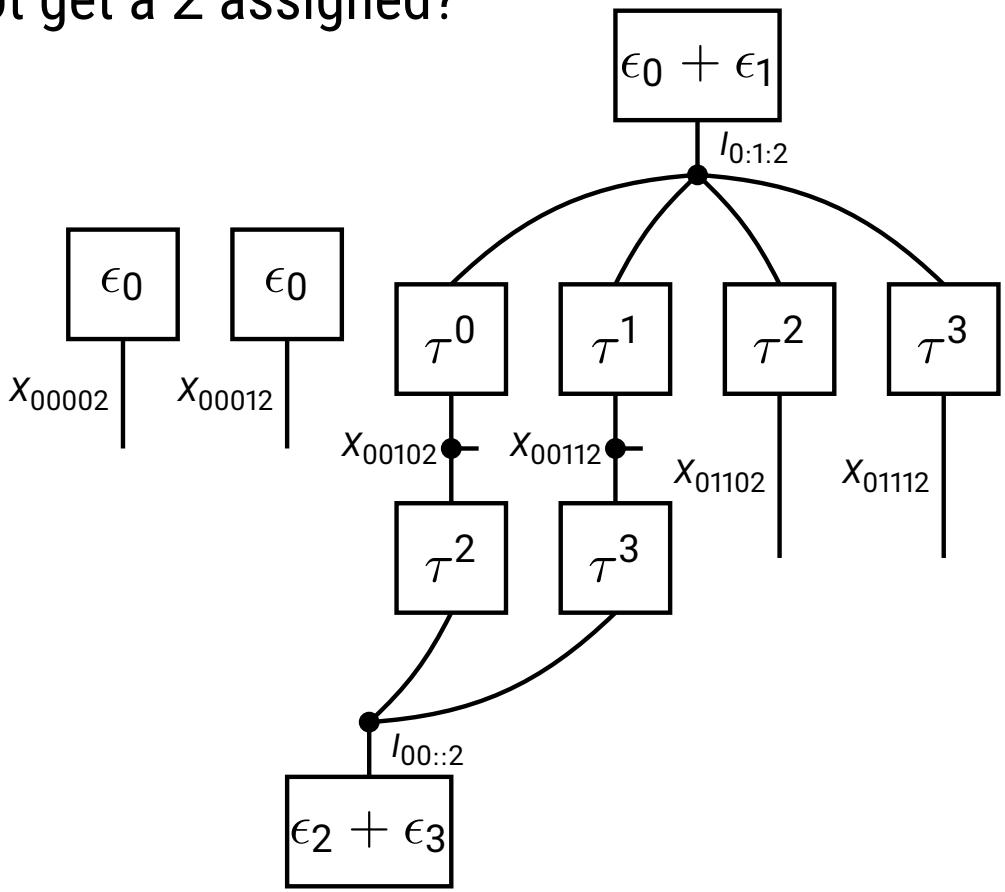


We need a contraction of four constraint matrices, to continue!

Limitations

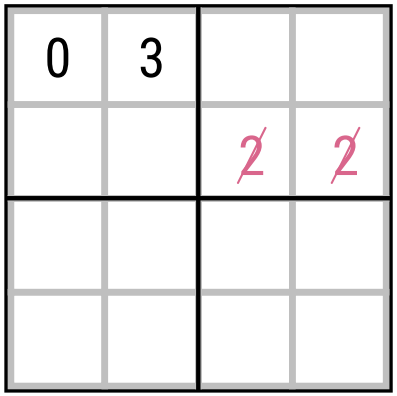
Example: Which cells cannot get a 2 assigned?

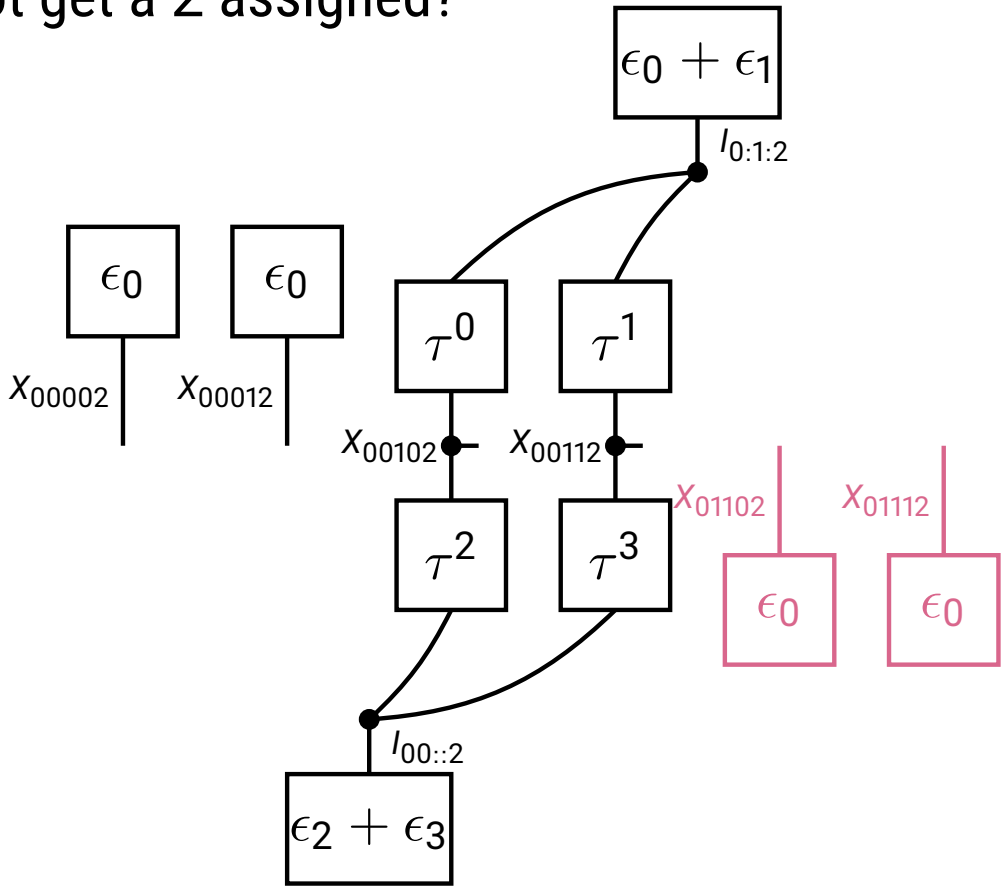




Limitations

Example: Which cells cannot get a 2 assigned?





Result: Two cells can be excluded for 2.

Outline

- Constraint Satisfaction Problems
- Sudoku
- Generalized unit propagation
- Graph Coloring
- Backtracking search

Example: Graph Coloring

Graph Coloring Problem

Given a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and m colors, find a coloring of the nodes, such that adjacent nodes have different colors.

Example: Paint the states of australia

- How many colors are required?
- How many possible colorings can be done?

Representation of Graph Coloring

We model the problem:

- Enumerate the colors by the state set $[m] = \{0, \ldots, m-1\}$
- For each node $v \in \mathcal{V}$ we define a variable X_v

To each edge $e = \{v_0, v_1\}$ the "different" constraint matrix

$$\tau^{\mathrm{diff},\{v_0,v_1\}}\left[X_{v_0}, X_{v_1}\right] = \mathbb{I}\left[X_{v_0}, X_{v_1}\right] - \delta\left[X_{v_0}, X_{v_1}\right] .$$

with off-diagonal support enforces different color constraint.

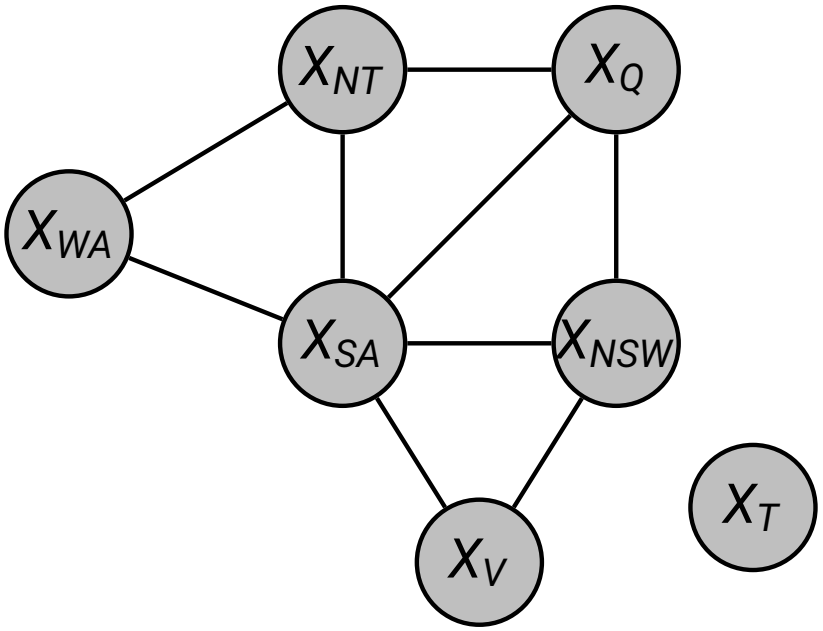
$$\tau^{\{v_0,v_1\}}\left[X_{v_0}, X_{v_1}\right] = \begin{pmatrix} 0 & 1 & \cdots & 1 \\ 1 & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & 1 \\ 1 & \cdots & 1 & 0 \end{pmatrix} \left[X_{v_0}, X_{v_1}\right]$$

Example: Painting states of australia

Let us paint the states of australia with 3 colors:

- 7 color variables $X.$ of leg dimension $m = 3$
- 9 edges between neighbored states v_0 and v_1, which are decorated by the 3x3 constraint matrix

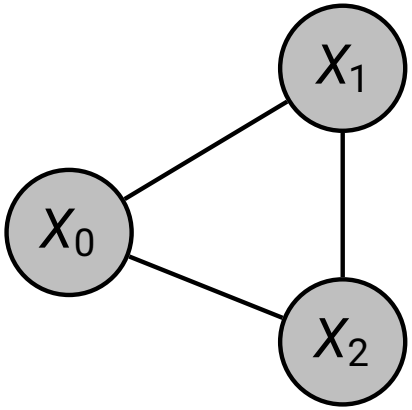
$$\tau^{\{v_0,v_1\}}\,[X_{v_0}, X_{v_1}] = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix} [X_{v_0}, X_{v_1}]$$



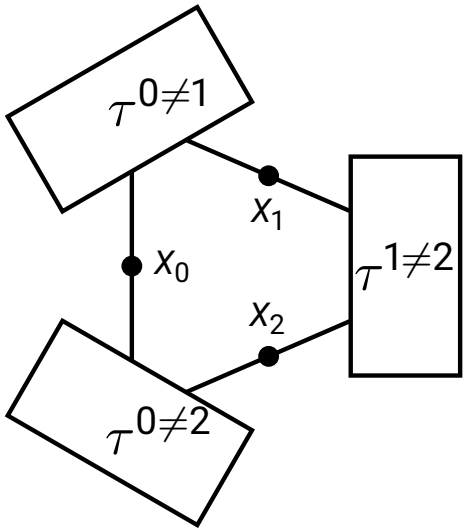
Example: Cycle of three nodes

Question: Is a cycle of three nodes colorable by two colors?

Consider a cycle of three nodes:



When colorable then the following holds:



$\neq \quad 0$

Cannot detect the unsatisfiability by contractions of two matrices.
In general: Cycles with odd numbers need three colors, of two need two colors.

Example: Cycle of three nodes

Let us allow for three colors (i.e. $m_0 = m_1 = m_2 = 3$). The contraction of the three "different" constraints is non-vanishing, and we can read off the solutions:

$$\left\langle \tau^{0\neq 1}\left[X_0, X_1\right], \tau^{0\neq 2}\left[X_0, X_2\right], \tau^{1\neq 2}\left[X_1, X_2\right]\right\rangle_{[X_0,X_1,X_2]} =$$

$$\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}_{[X_0, X_1, X_2]}$$

Interpretation: The contracted tensor has 6 non-vanishing entries, which correspond to the 6 possible colorings of the three nodes by three colors.

Disentangling properties of the "different" constraint

When contracting the "different" constraint with a vector, we get

$$\mathbb{I}_{\neq 0} \circ \left\langle V[X_{v_0}], \tau^{\{v_0,v_1\}}[X_{v_0}, X_{v_1}] \right\rangle_{[X_{v_1}]}$$
$$= \begin{cases} \mathbb{I}[X_{v_1}] - \epsilon_y [X_{v_1}] & \text{if } V[X_{v_0}] \text{ only at } y \text{ supported} \\ \mathbb{I}[X_{v_1}] & \text{else} \end{cases}.$$

The "different" constraint has weakest disentangling properties:

Disentangling only possible, when the vector is basis (i.e. the color of one node is known).

Consequence: Unit propagation helps only, when the color of one node v_0 is known:

But where to find the basis vectors necessary for disentangling the color constraints?

Outline

- Constraint Satisfaction Problems
- Sudoku
- Generalized unit propagation
- Graph Coloring
- Backtracking search

Idea of backtracking

Problem: Some constraints do not disentangle. This might be due to the fact, that multiple solutions are available (which do not build product sets).

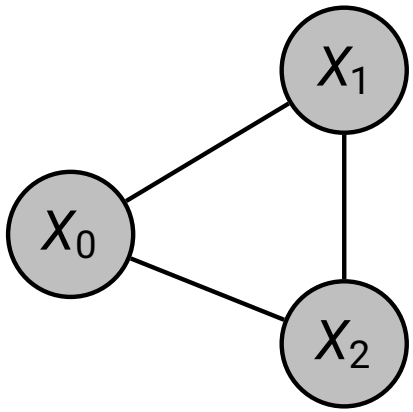
Solution: Backtracking search

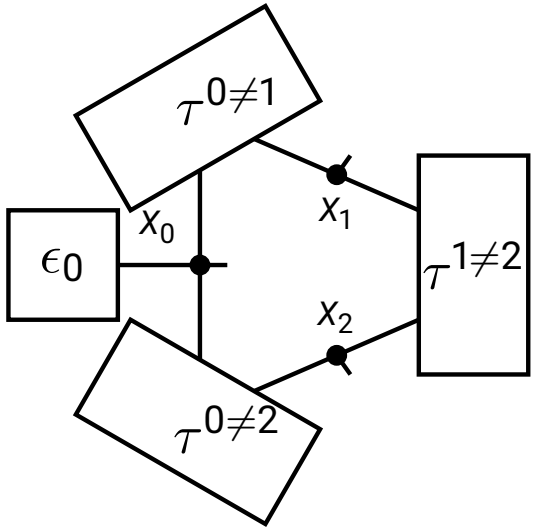
- Guess an assignment to a variable, and add the one-hot encoding $\epsilon_i\,[X_v]$.
- Any constraint tensor to an edge e with $v \in e$ disentangles (maybe more by unit propagation).
- If an inconsistency is detected (i.e. a vanishing constraint tensor), guess an alternative assignment.
- Iterate until a non-vanishing elementary constraint network is reached, from which (possibly multiple) solutions can be constructed by local solutions.

Example: Coloring a cycle of three nodes

To show that a cycle of three nodes is not colorable by two colors, we can use backtracking:

- Guess X_0 to be 0, and get an inconsistency.
-

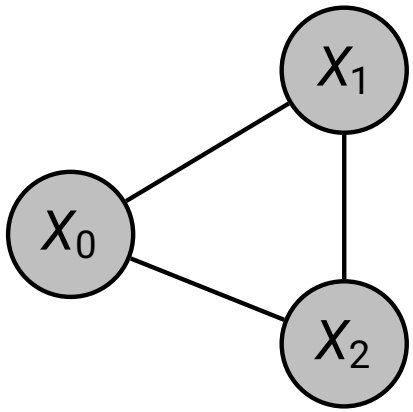


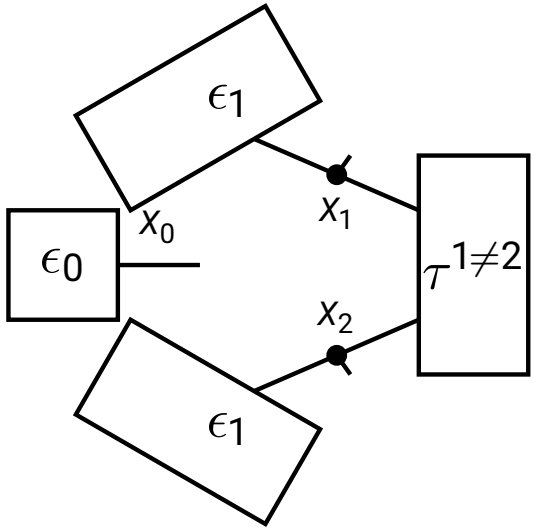


Example: Coloring a cycle of three nodes

To show that a cycle of three nodes is not colorable by two colors, we can use backtracking:

- Guess X_0 to be 0, and get an inconsistency.
-

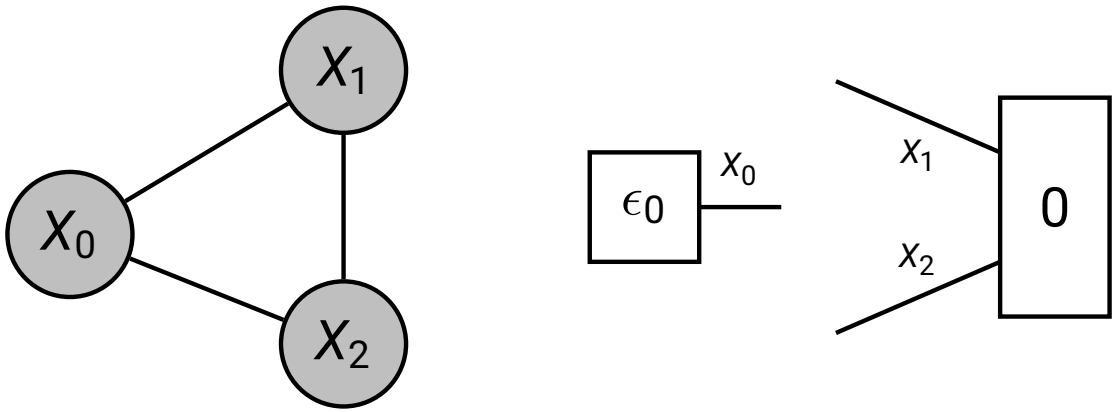




Example: Coloring a cycle of three nodes

To show that a cycle of three nodes is not colorable by two colors, we can use backtracking:

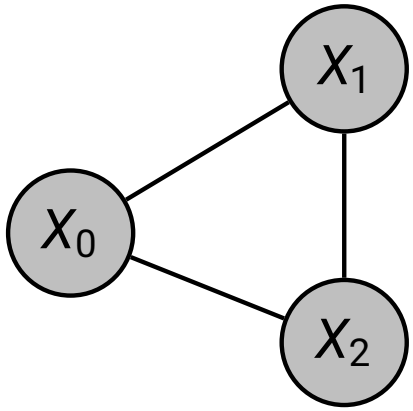
- Guess X_0 to be 0, and get an inconsistency.
-

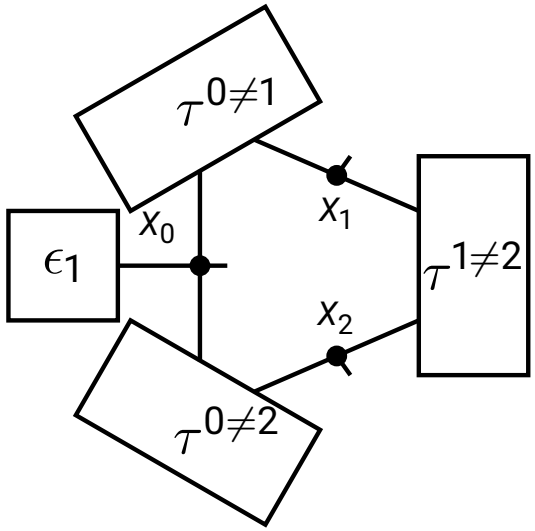


Example: Coloring a cycle of three nodes

To show that a cycle of three nodes is not colorable by two colors, we can use backtracking:

- Guess X_0 to be 0, and get an inconsistency.
- Guess X_0 to be 1, and get an inconsistency.

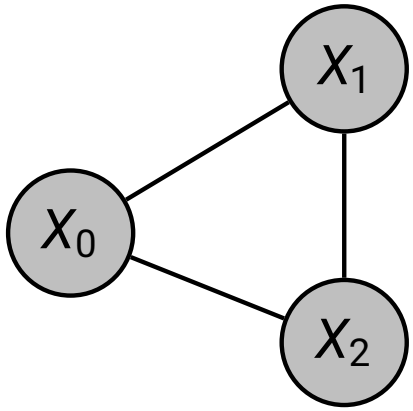


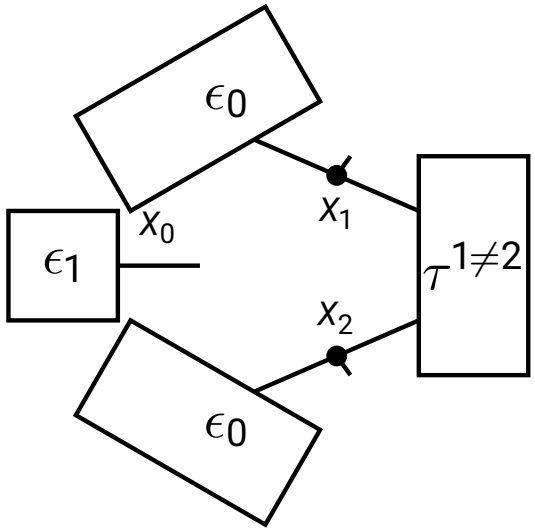


Example: Coloring a cycle of three nodes

To show that a cycle of three nodes is not colorable by two colors, we can use backtracking:

- Guess X_0 to be 0, and get an inconsistency.
- Guess X_0 to be 1, and get an inconsistency.

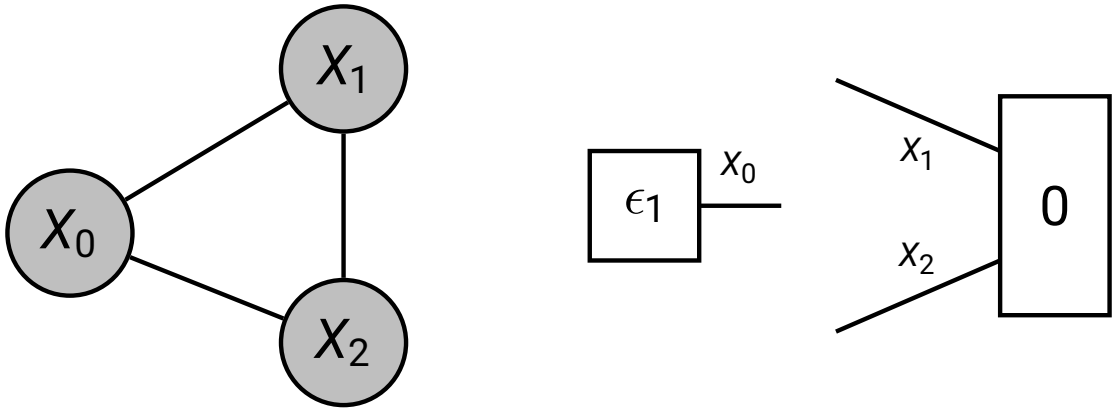




Example: Coloring a cycle of three nodes

To show that a cycle of three nodes is not colorable by two colors, we can use backtracking:

- Guess X_0 to be 0, and get an inconsistency.
- Guess X_0 to be 1, and get an inconsistency.



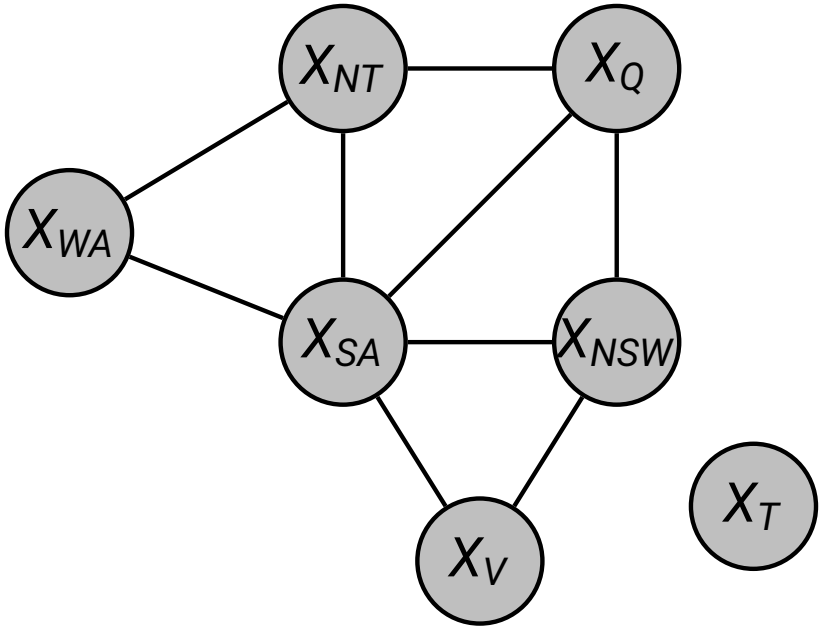
Backtracking algorithm

Require: Constraint network $\tau^{\mathcal{G}}$
Ensure: Elementary constraint network

Choose a node v in a non-unitary edge e
for $x_v \in [m_v]$ **do**
  $\tilde{\tau^{\mathcal{G}}} \leftarrow \tau^{\mathcal{G}} \cup \{\epsilon_{x_v}[X_v]\}$
  $\tilde{\tau^{\mathcal{G}}} \leftarrow \mathrm{Simplify}(\tilde{\tau^{\mathcal{G}}})$ (differing realization)
  if $\tilde{\tau^{\mathcal{G}}}$ vanishing **then**
    return *False*
  end if
  $\tilde{\tau^{\mathcal{G}}} \leftarrow \mathrm{Backtrack}(\tau^{\mathcal{G}} \cup \{\epsilon_{x_v}[X_v]\})$
  if $\tau^{\mathcal{G}}$ not *False* **then**
    return $\tau^{\mathcal{G}}$
  end if
end for
return *False*

Example: Painting australias states

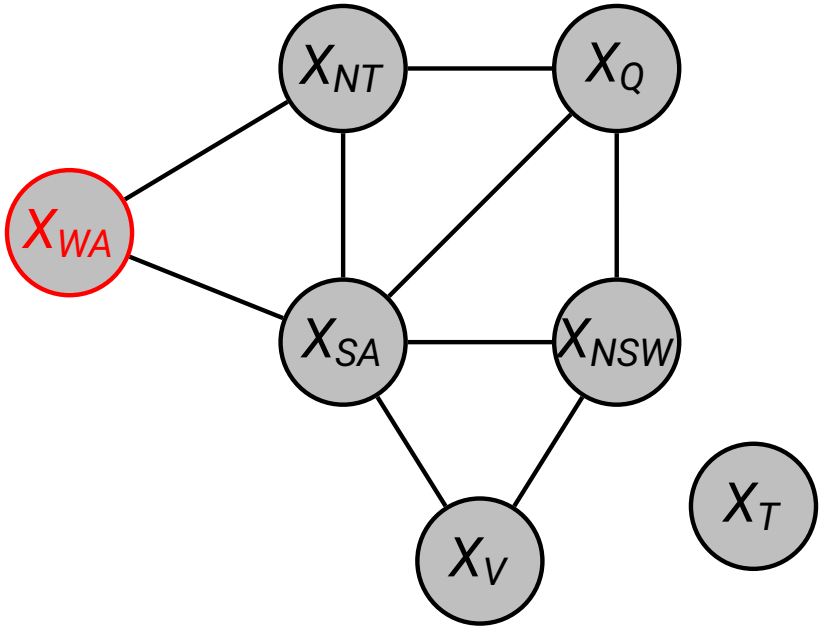
Take three colors: red, blue, green



-
-
-
-
-
-

Example: Painting australias states

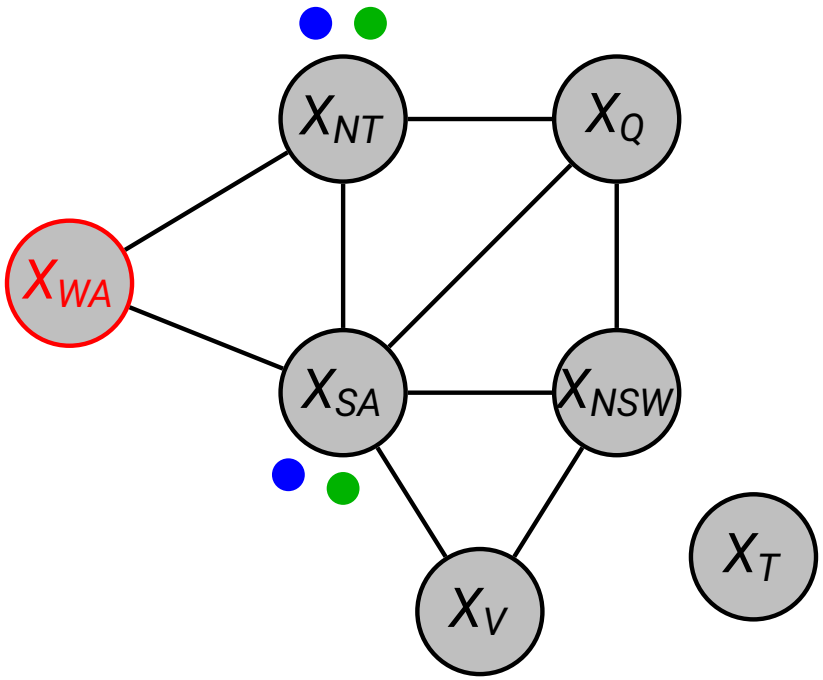
Take three colors: red, blue, green



- Guess X_{WA} to be red.
-
-
-
-
-

Example: Painting australias states

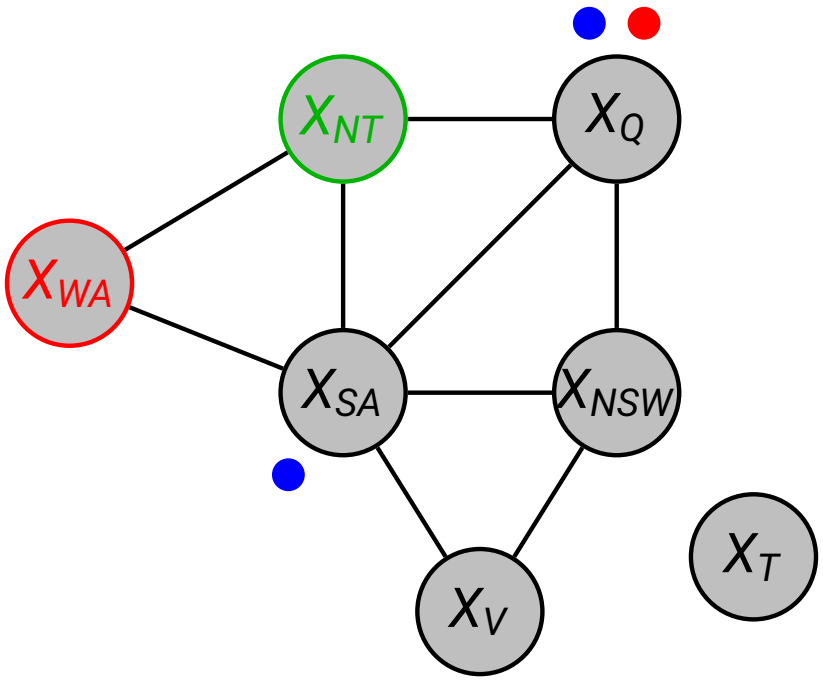
Take three colors: red, blue, green



- Guess X_{WA} to be red.
- GUP finds consequences for X_{NT} and X_{SA}: red or green
-
-
-
-

Example: Painting australias states

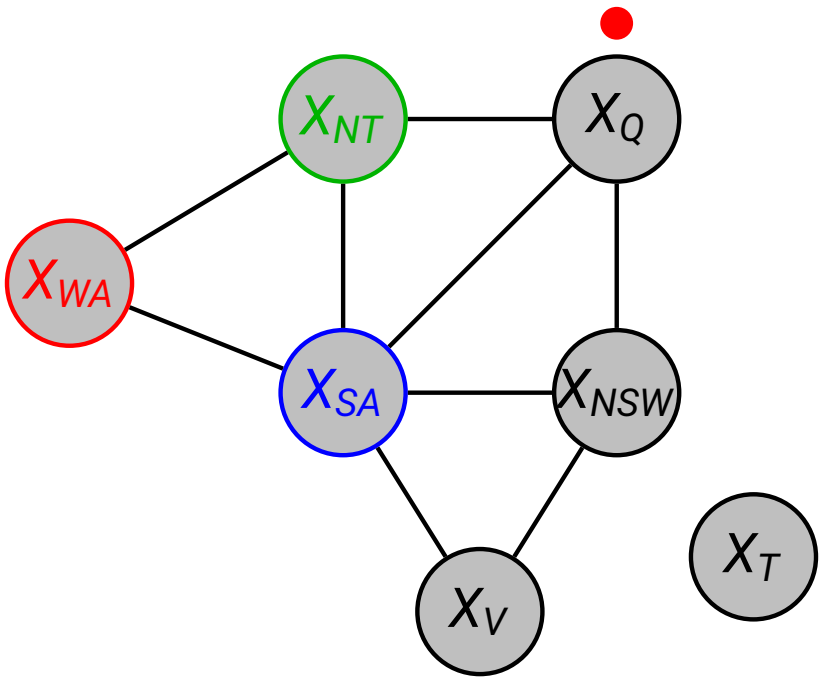
Take three colors: red, blue, green



- Guess X_{WA} to be red.
- GUP finds consequences for X_{NT} and X_{SA}: red or green
- Guess X_{NT} to be green.
-
-
-

Example: Painting australias states

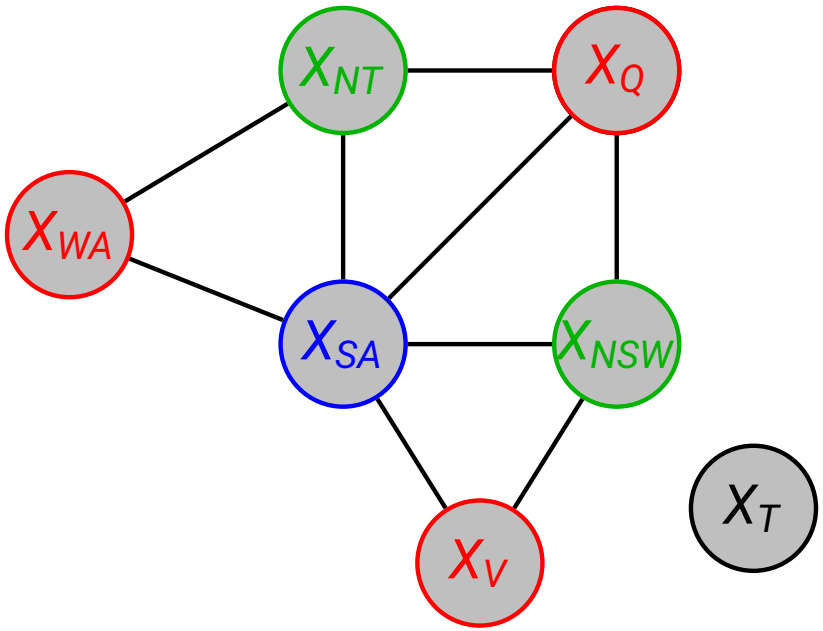
Take three colors: red, blue, green



- Guess X_{WA} to be red.
- GUP finds consequences for X_{NT} and X_{SA}: red or green
- Guess X_{NT} to be green.
- GUP dermines X_{SA} to be blue.
-
-

Example: Painting australias states

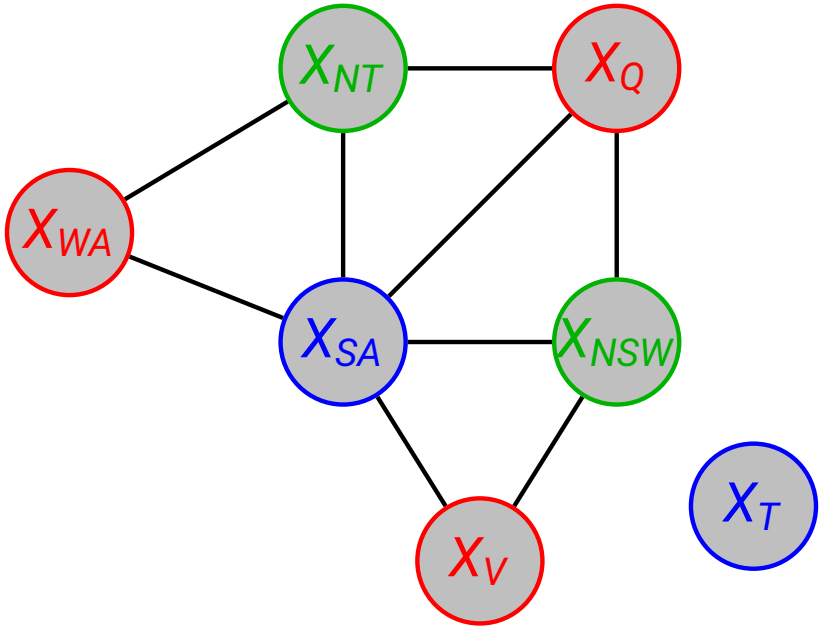
Take three colors: red, blue, green



- Guess X_{WA} to be red.
- GUP finds consequences for X_{NT} and X_{SA}: red or green
- Guess X_{NT} to be green.
- GUP dermines X_{SA} to be blue.
- GUP determines also X_Q, X_{NSW}, X_V.
-

Example: Painting australias states

Take three colors: red, blue, green



- Guess X_{WA} to be red.
- GUP finds consequences for X_{NT} and X_{SA}: red or green
- Guess X_{NT} to be green.
- GUP dermines X_{SA} to be blue.
- GUP determines also X_Q, X_{NSW}, X_V.
- The variable X_T is disconnected and can be chosen arbitrarly.